VEHICLE INSURANCE MANAGEMENT SYSTEM

API Documentation

Version 1.0 | March 2026

Base URL: https://localhost:7257/api

Overview

This document describes all the REST API endpoints available in the Vehicle Insurance Management System (VIMS). The API is built with ASP.NET Core and is organized into five controller groups: Admin, Agent, Auth, ClaimsOfficer, and Customer.

Base URL

```
https://<host>/api
```

Authentication

Most endpoints require a valid JWT Bearer token. After logging in, include the token in every request header like this:

```
Authorization: Bearer <your_token_here>
```

Response Format

All responses are JSON. Successful responses return the requested data. Failed responses include an error message. Common HTTP status codes used:

200 OK	Request worked. Returns data or a success message.
201 Created	A new resource was created successfully.
400 Bad Request	Something wrong with the request body or parameters.
401 Unauthorized	No token provided or the token is invalid/expired.
403 Forbidden	Token is valid but the user does not have the right role.
404 Not Found	The requested resource does not exist.
500 Server Error	Something went wrong on the server side.

All Endpoints at a Glance

Admin

Method	Endpoint	Description
POST	`/api/Admin/createAgent`	Create a new Agent account
POST	`/api/Admin/createClaimsOfficer`	Create a new Claims Officer account
POST	`/api/Admin/createPolicyPlan`	Create a new insurance plan

Method	Endpoint	Description
GET	/api/Admin/policy-plans	Get all policy plans
GET	/api/Admin/policy-plan/{id}	Get a specific policy plan by ID
GET	/api/Admin/users	Get all users in the system
GET	/api/Admin/claims	Get all claims
GET	/api/Admin/payments	Get all payment records
GET	/api/Admin/policies	Get all policies
PUT	/api/Admin/deactivate/{id}	Deactivate a user account
PUT	/api/Admin/activate/{id}	Activate a user account
GET	/api/Admin/audit-logs	Get all audit log entries

Agent

Method	Endpoint	Description
PUT	/api/Agent/vehicle-application/{id}/review	Approve or reject a vehicle application
GET	/api/Agent/pending-applications	Get applications with Pending status
GET	/api/Agent/customers	Get list of customers
GET	/api/Agent/applications	Get all applications assigned to the agent

Auth

Method	Endpoint	Description
POST	/api/Auth/customer/register	Register a new customer account
POST	/api/Auth/login	Log in and get a JWT token
POST	/api/Auth/admin/register	Register a new admin account
PUT	/api/Auth/change-password	Change current user password
GET	/api/Auth/forgot-password/security-question/{email}	Get security question for password reset
POST	/api/Auth/forgot-password/reset	Reset password using security answer
POST	/api/Auth/set-security-question	Set or update security question and answer

ClaimsOfficer

Method	Endpoint	Description
GET	/api/ClaimsOfficer/claim/{id}	Get details of a specific claim
GET	/api/ClaimsOfficer/all-claims	Get all claims in the system
GET	/api/ClaimsOfficer/my-claims	Get claims assigned to this officer
POST	/api/ClaimsOfficer/decide/{claimId}	Submit approve or reject decision on a claim

Customer

Method	Endpoint	Description
GET	/api/Customer/invoice/download/{paymentId}	Download invoice PDF for a payment
GET	/api/Customer/policy/{policyId}	Get full details of a policy
GET	/api/Customer/claim/{id}	Get details of a specific claim
GET	/api/Customer/policy/{policyId}/status	Get current status of a policy
GET	/api/Customer/policy/{policyId}/payment-status	Check if current year is paid
GET	/api/Customer/policy/{policyId}/years	Get year-by-year breakdown of a policy
GET	/api/Customer/all-policy-plans	Get all active policy plans
POST	/api/Customer/vehicle-application	Submit a new vehicle application
GET	/api/Customer/my-applications	Get all vehicle applications by this customer
POST	/api/Customer/calculate-quote	Calculate a premium quote without applying
GET	/api/Customer/my-policies	Get all policies belonging to this customer
POST	/api/Customer/pay-annual/{policyId}	Pay the annual premium for a policy
POST	/api/Customer/renew/{policyId}	Renew a policy for another year
POST	/api/Customer/claim/submit	Submit a new insurance claim
GET	/api/Customer/claims/my	Get all claims filed by this customer
POST	/api/Customer/policy/cancel/{policyId}	Cancel an active policy
GET	/api/Customer/payments/my	Get all payments made by this customer
POST	/api/Customer/transfer/initiate	Initiate a policy transfer to another customer

Method	Endpoint	Description
GET	`/api/Customer/transfer/incoming`	Get incoming transfer requests
GET	`/api/Customer/transfer/outgoing`	Get outgoing transfer requests
POST	`/api/Customer/transfer/{transferId}/accept`	Accept an incoming transfer request
POST	`/api/Customer/transfer/{transferId}/reject`	Reject an incoming transfer request

Admin Endpoints

All Admin endpoints require a JWT token with the Admin role (Role = 3). Requests without the correct role will get a 403 Forbidden response.

POST /api/Admin/createAgent

Create a new Agent user account.

Auth Required: JWT required — Admin only

Creates a new user account with the Agent role. The admin provides all the details. The password is hashed before saving.

Request Body (JSON)

```
{
  "fullName": "Rajesh Kumar",
  "email": "rajesh.agent@vims.com",
  "password": "SecurePass@123"
}
```

Responses

201 Created	Agent account created successfully. Returns the new user object.
400 Bad Request	Email already exists or required fields are missing.
401/403	Token missing or not Admin role.

POST /api/Admin/createClaimsOfficer

Create a new Claims Officer user account.

Auth Required: JWT required — Admin only

Creates a new user account with the Claims Officer role. Same structure as creating an Agent.

Request Body (JSON)

```
{
  "fullName": "Priya Sharma",
  "email": "priya.officer@vims.com",
  "password": "SecurePass@456"
}
```

Responses

201 Created	Claims Officer account created. Returns the new user object.
400 Bad Request	Email already exists or required fields are missing.
401/403	Token missing or not Admin role.

POST /api/Admin/createPolicyPlan

Create a new insurance policy plan.

Auth Required: JWT required — Admin only

Adds a new PolicyPlan to the system. Once created and set to active, customers can see it in the Explore Plans page.

Request Body (JSON)

```
{
  "planName": "Comprehensive Plus",
  "policyType": "Comprehensive",
  "basePremium": 8500.00,
  "maxCoverageAmount": 500000.00,
  "policyDurationMonths": 12,
  "applicableVehicleType": "Car",
  "coversOwnDamage": true,
  "coversTheft": true,
  "coversThirdParty": true,
  "deductibleAmount": 2000.00,
  "engineProtectionAvailable": true,
  "roadsideAssistanceAvailable": true,
  "zeroDepreciationAvailable": true,
  "status": 1
}
```

Responses

201 Created	Plan created successfully. Returns the new plan object.
400 Bad Request	Validation failed — check required fields and data types.
401/403	Token missing or not Admin role.

Notes

- Status 1 = Active (visible to customers). Status 0 = Inactive (hidden).
- ApplicableVehicleType should match the vehicle types customers register (e.g., Car, Bike, Truck).

GET `/api/Admin/policy-plans`

Get all policy plans — both active and inactive.

Auth Required: JWT required — Admin only

Returns the full list of all PolicyPlans in the system regardless of status. Customers only see active plans on the Explore Plans page, but admins can see everything here.

Responses

200 OK	Returns array of all PolicyPlan objects.
401/403	Token missing or not Admin role.

GET `/api/Admin/policy-plan/{id}`

Get a single policy plan by its ID.

Auth Required: JWT required — Admin only

Path Parameters

id	int — The PlanId of the policy plan to retrieve.

Responses

200 OK	Returns the PolicyPlan object.
404 Not Found	No plan found with that ID.
401/403	Token missing or not Admin role.

GET `/api/Admin/users`

Get all users registered in the system.

Auth Required: JWT required — Admin only

Returns every user regardless of role. Useful for the Admin Dashboard user management page.

Responses

200 OK	Returns array of user objects (UserId, FullName, Email, Role, IsActive).
401/403	Token missing or not Admin role.

GET `/api/Admin/claims`

Get all insurance claims across the entire system.

Auth Required: JWT required — Admin only

Returns every Claim filed by any customer. Includes claim status, type, approved amount, and linked policy info.

Responses

200 OK	Returns array of all Claim objects.
401/403	Token missing or not Admin role.

GET `/api/Admin/payments`

Get all payment records in the system.

Auth Required: JWT required — Admin only

Returns all Payments made across all policies. Used for the financial ledger view in the Admin Dashboard.

Responses

200 OK	Returns array of all Payment objects including Amount, PaymentDate, Status, PaymentMethod, TransactionReference.
401/403	Token missing or not Admin role.

GET `/api/Admin/policies`

Get all policies in the system.

Auth Required: JWT required — Admin only

Returns all Policies regardless of customer or status. Admins can see the full picture of issued policies.

Responses

200 OK	Returns array of all Policy objects.
401/403	Token missing or not Admin role.

PUT `/api/Admin/deactivate/{id}`

Deactivate a user account.

Auth Required: JWT required — Admin only

Sets IsActive = false on the specified user. A deactivated user cannot log in. Their data is not deleted.

Path Parameters

id	int — UserId of the account to deactivate.

Responses

200 OK	User deactivated successfully.
404 Not Found	No user found with that ID.
401/403	Token missing or not Admin role.

PUT `/api/Admin/activate/{id}`

Activate a user account.

Auth Required: JWT required — Admin only

Sets IsActive = true on the specified user. Use this to re-enable accounts that were deactivated.

Path Parameters

id	int — UserId of the account to activate.

Responses

200 OK	User activated successfully.
404 Not Found	No user found with that ID.
401/403	Token missing or not Admin role.

GET `/api/Admin/audit-logs`

Get all AuditLog entries.

Auth Required: JWT required — Admin only

Returns every entry from the AuditLog table. Each entry records who did what and when. Logs are read-only and cannot be deleted.

Responses

200 OK	Returns array of AuditLog objects (Action, Category, UserId, Email, Role, EntityName, EntityId, Details, Timestamp).
401/403	Token missing or not Admin role.

Notes

- AuditLog entries are immutable — there is no endpoint to delete or edit them.
- Use query parameters if filtering by date range or category is supported on the server.

Agent Endpoints

All Agent endpoints require a JWT token with the Agent role (Role = 1).

PUT /api/Agent/vehicle-application/{id}/review

Approve or reject a vehicle application.

Auth Required: JWT required — Agent only

This is the main action agents perform. When approved, the system automatically creates a Vehicle record and a Policy record (Status = Pending Payment). PricingService calculates all financial fields on the policy.

Path Parameters

id	int — VehicleApplicationId to review.

Request Body (JSON)

```
{
  "decision": "Approved",
  "rejectionReason": null
}

// For rejection:
{
  "decision": "Rejected",
  "rejectionReason": "Registration number mismatch in documents"
}
```

Responses

200 OK	Decision recorded. If approved, Vehicle and Policy records are created.
400 Bad Request	Invalid decision value or rejection without a reason.
404 Not Found	Application not found or not assigned to this agent.
401/403	Token missing or not Agent role.

Notes

- Agents can only review applications assigned to them (AssignedAgentId = their UserId).
- Rejection requires a RejectionReason — cannot be left empty.
- On approval, PricingService sets PremiumAmount, IDV, InvoiceAmount, StartDate, EndDate, CurrentYearEndDate, CurrentYearNumber, and SelectedYears on the new Policy.

GET /api/Agent/pending-applications

Get vehicle applications with Pending status assigned to this agent.

Auth Required: JWT required — Agent only

Returns only the applications that are still waiting for review (Status = Pending) and are assigned to the logged-in agent.

Responses

200 OK	Returns array of pending VehicleApplication objects including uploaded

	VehicleDocuments.
401/403	Token missing or not Agent role.

GET `/api/Agent/customers`

Get a list of customers.

Auth Required: JWT required — Agent only

Returns user accounts with the Customer role. Useful for the agent to look up customer details.

Responses

200 OK	Returns array of customer user objects.
401/403	Token missing or not Agent role.

GET `/api/Agent/applications`

Get all vehicle applications assigned to this agent.

Auth Required: JWT required — Agent only

Returns all VehicleApplications assigned to the logged-in agent regardless of status (Pending, Approved, Rejected). Good for history view.

Responses

200 OK	Returns array of VehicleApplication objects.
401/403	Token missing or not Agent role.

Auth Endpoints

Auth endpoints handle registration, login, and password management. Most of these do not require a token. The ones that do are clearly marked.

POST /api/Auth/customer/register

Register a new customer account.

Auth Required: No auth required

Creates a new User with Role = Customer (0) and IsActive = true. AuthService validates the CAPTCHA token before creating the account. Password and security answer are hashed before saving.

Request Body (JSON)

```
{
  "fullName": "Ananya Patel",
  "email": "ananya@example.com",
  "password": "MyPass@2026",
  "captchaToken": "<captcha_response_token>",
  "securityQuestion": "What is your mother's maiden name?",
  "securityAnswer": "Sharma"
}
```

Responses

201 Created	Account created. Returns the new user's basic details.
400 Bad Request	Email already exists, CAPTCHA failed, or required fields are missing.

Notes

- CAPTCHA validation happens first — if it fails, the account is not created.
- Role is automatically set to Customer. You cannot self-register as an Agent or Admin.

POST /api/Auth/login

Log in and receive a JWT token.

Auth Required: No auth required

Validates the user's email, password, and CAPTCHA. If everything checks out and the account is active, returns a signed JWT token. The token contains the user's ID, email, and role.

Request Body (JSON)

```
{
  "email": "ananya@example.com",
  "password": "MyPass@2026",
  "captchaToken": "<captcha_response_token>"
}
```

Responses

200 OK	Login successful. Returns JWT token and basic user info (UserId, FullName, Email, Role).

400 Bad Request	CAPTCHA failed.
401 Unauthorized	Wrong email or password.
403 Forbidden	Account is inactive (IsActive = false). Contact admin.

Notes

- Store the returned JWT token in the frontend and attach it to all protected requests.
- The Angular HTTP Interceptor handles this automatically.

POST `/api/Auth/admin/register`

Register a new Admin account.

Auth Required: No auth required (typically restricted at network/environment level)

Creates a User with Role = Admin (3). This endpoint should be secured or disabled in production — typically used for initial setup only.

Request Body (JSON)

```
{
  "fullName": "System Admin",
  "email": "admin@vims.com",
  "password": "AdminSecure@2026"
}
```

Responses

201 Created	Admin account created.
400 Bad Request	Email already exists or validation failed.

Notes

- This endpoint should ideally be disabled or protected in production environments.

PUT `/api/Auth/change-password`

Change the password for the currently logged-in user.

Auth Required: JWT required — any role

Allows a logged-in user to change their own password. They must provide the current password to confirm it is them.

Request Body (JSON)

```
{
  "currentPassword": "MyPass@2026",
  "newPassword": "NewPass@2027"
}
```

Responses

200 OK	Password changed successfully.
400 Bad Request	Current password is wrong or new password does not meet requirements.
401 Unauthorized	Token missing or expired.

GET /api/Auth/forgot-password/security-question/{email}

Get the security question for a given email address.

Auth Required: No auth required

First step of the forgot password flow. The frontend calls this with the user's email to get their security question, which is then shown to the user to answer.

Path Parameters

email	string — The email address of the account to recover.

Responses

200 OK	Returns the SecurityQuestion string for that email.
404 Not Found	No account found with that email.

Notes

- Returns only the question text — not the answer or any other user data.

POST /api/Auth/forgot-password/reset

Reset password using the security question answer.

Auth Required: No auth required

Second step of the forgot password flow. The user submits their email, security answer, and new password. AuthService validates the answer hash and updates the password.

Request Body (JSON)

```
{
  "email": "ananya@example.com",
  "securityAnswer": "Sharma",
  "newPassword": "ResetPass@2027"
}
```

Responses

200 OK	Password reset successfully.
400 Bad Request	Security answer is wrong or new password validation failed.
404 Not Found	No account found with that email.

POST /api/Auth/set-security-question

Set or update the security question and answer for the logged-in user.

Auth Required: JWT required — any role

Lets a logged-in user set or update their SecurityQuestion and SecurityAnswer. The answer is hashed before being saved.

Request Body (JSON)

```
{
```

```
  "securityQuestion": "What city were you born in?",
  "securityAnswer": "Mumbai"
}
```

Responses

200 OK	Security question and answer saved.
400 Bad Request	Missing fields.
401 Unauthorized	Token missing or expired.

ClaimsOfficer Endpoints

All ClaimsOfficer endpoints require a JWT token with the Claims Officer role (Role = 2).

GET `/api/ClaimsOfficer/claim/{id}`

Get full details of a specific claim.

Auth Required: JWT required — Claims Officer only

Returns all information about a claim including ClaimDocuments, linked policy, customer info, and current status.

Path Parameters

id	int — ClaimId to retrieve.

Responses

200 OK	Returns the Claim object with full details and ClaimDocuments.
404 Not Found	No claim found with that ID.
401/403	Token missing or not Claims Officer role.

GET `/api/ClaimsOfficer/all-claims`

Get all claims in the system.

Auth Required: JWT required — Claims Officer only

Returns every Claim regardless of which officer it is assigned to. Useful for the claims queue view where officers can pick up unassigned claims.

Responses

200 OK	Returns array of all Claim objects.
401/403	Token missing or not Claims Officer role.

GET `/api/ClaimsOfficer/my-claims`

Get claims assigned to the logged-in officer.

Auth Required: JWT required — Claims Officer only

Returns only the claims where ClaimsOfficerId matches the logged-in officer's UserId. These are the claims this officer has taken action on.

Responses

200 OK	Returns array of Claim objects assigned to this officer.
401/403	Token missing or not Claims Officer role.

POST `/api/ClaimsOfficer/decide/{claimId}`

Submit a decision (approve or reject) on a claim.

Auth Required: JWT required — Claims Officer only

The main action for claims officers. Sets the claim status to Approved or Rejected, records the officer's ID, decision type, and approved amount (if approving). ClaimsService validates that ApprovedAmount does not exceed MaxCoverageAmount minus DeductibleAmount.

Path Parameters

claimId	int — ClaimId to decide on.

Request Body (JSON)

```
// Approval:
{
  "decision": "Approved",
  "approvedAmount": 45000.00,
  "decisionType": "Full repair cost covered after deductible"
}

// Rejection:
{
  "decision": "Rejected",
  "approvedAmount": null,
  "decisionType": "Claim type not covered under the selected plan"
}
```

Responses

200 OK	Decision recorded. Claim status updated to Approved or Rejected.
400 Bad Request	ApprovedAmount exceeds plan limit, or required fields missing.
404 Not Found	Claim not found.
401/403	Token missing or not Claims Officer role.

Notes

- ApprovedAmount cannot exceed (MaxCoverageAmount - DeductibleAmount) from the linked PolicyPlan.
- DecisionType is a text description of the decision — not an enum.
- The logged-in officer's UserId is automatically saved to ClaimsOfficerId on the Claim.

Customer Endpoints

All Customer endpoints require a JWT token with the Customer role (Role = 0). Customers can only access their own data — they cannot see other customers' policies, claims, or payments.

GET `/api/Customer/invoice/download/{paymentId}`

Download the invoice for a completed payment.

Auth Required: JWT required — Customer only (own payments)

Returns a downloadable invoice file for the specified payment. The payment must belong to the logged-in customer.

Path Parameters

paymentId	int — PaymentId to download invoice for.

Responses

200 OK	Returns invoice file (PDF or similar).
404 Not Found	Payment not found or does not belong to this customer.
401/403	Token missing or not Customer role.

GET `/api/Customer/policy/{policyId}`

Get full details of a specific policy.

Auth Required: JWT required — Customer only (own policies)

Returns everything about a policy including vehicle info, plan details, payment history, claim count, and current year tracking.

Path Parameters

policyId	int — PolicyId to retrieve.

Responses

200 OK	Returns full Policy object.
404 Not Found	Policy not found or does not belong to this customer.
401/403	Token missing or not Customer role.

GET `/api/Customer/claim/{id}`

Get details of a specific claim filed by this customer.

Auth Required: JWT required — Customer only (own claims)

Returns the Claim object with its current status, ClaimDocuments, and decision details if it has been decided.

Path Parameters

id	int — ClaimId to retrieve.

Responses

200 OK	Returns Claim object with documents and status.
404 Not Found	Claim not found or does not belong to this customer.
401/403	Token missing or not Customer role.

GET `/api/Customer/policy/{policyId}/status`

Get the current status of a policy.

Auth Required: JWT required — Customer only (own policies)

Returns just the Status value of the policy. Useful for quick status checks without loading the full policy object.

Path Parameters

policyId	int — PolicyId to check.

Responses

200 OK	Returns the policy status (e.g., Active, Expired, Cancelled).
404 Not Found	Policy not found.
401/403	Token missing or not Customer role.

GET `/api/Customer/policy/{policyId}/payment-status`

Check if the current year's premium is paid.

Auth Required: JWT required — Customer only (own policies)

Returns the IsCurrentYearPaid flag for the specified policy. The frontend uses this to show whether the customer needs to make a payment.

Path Parameters

policyId	int — PolicyId to check.

Responses

200 OK	Returns IsCurrentYearPaid (true or false).
404 Not Found	Policy not found.
401/403	Token missing or not Customer role.

GET `/api/Customer/policy/{policyId}/years`

Get year-by-year breakdown of a multi-year policy.

Auth Required: JWT required — Customer only (own policies)

Returns the year tracking details for the policy — CurrentYearNumber, CurrentYearEndDate, SelectedYears, and payment status per year. Useful for displaying the policy timeline.

Path Parameters

policyId	int — PolicyId to check.

Responses

200 OK	Returns year breakdown data.
404 Not Found	Policy not found.
401/403	Token missing or not Customer role.

GET `/api/Customer/all-policy-plans`

Get all active policy plans.

Auth Required: JWT required — Customer only

Returns all PolicyPlans with Status = Active. This is the authenticated version of the Explore Plans data — used inside the Customer Dashboard when submitting a vehicle application.

Responses

200 OK	Returns array of active PolicyPlan objects.
401/403	Token missing or not Customer role.

POST `/api/Customer/vehicle-application`

Submit a new vehicle application.

Auth Required: JWT required — Customer only

Creates a new VehicleApplication. PricingService calculates the InvoiceAmount on the server based on the selected plan and vehicle details. The application is assigned to an available Agent automatically.

Request Body (JSON)

```
{
  "registrationNumber": "MH12AB1234",
  "make": "Maruti",
  "model": "Swift",
  "year": 2021,
  "fuelType": "Petrol",
  "vehicleType": "Car",
  "planId": 3,
  "kilometersDriven": 25000,
  "policyYears": 2,
  "documents": [
    { "documentType": "RC Book", "filePath": "uploads/rc_mh12ab1234.pdf" },
    { "documentType": "ID Proof", "filePath": "uploads/id_ananya.pdf" }
  ]
}
```

Responses

201 Created	Application submitted. Returns the new VehicleApplication object.
400 Bad Request	RegistrationNumber already exists or required fields missing.
401/403	Token missing or not Customer role.

Notes

- InvoiceAmount is calculated by PricingService on the server — do not send it in the request.
- The application starts with Status = Pending and gets auto-assigned to an Agent.
- RegistrationNumber must be unique across all Vehicles in the system.

GET /api/Customer/my-applications

Get all vehicle applications submitted by this customer.

Auth Required: JWT required — Customer only

Returns all VehicleApplications where CustomerId matches the logged-in user. Includes all statuses so customers can track their pending, approved, and rejected applications.

Responses

200 OK	Returns array of VehicleApplication objects.
401/403	Token missing or not Customer role.

POST /api/Customer/calculate-quote

Calculate a premium quote without submitting an application.

Auth Required: JWT required — Customer only

Lets customers see how much their premium would be before actually applying. PricingService calculates the estimate based on vehicle and plan details. Nothing is saved.

Request Body (JSON)

```
{
  "planId": 3,
  "vehicleType": "Car",
  "year": 2021,
  "kilometersDriven": 25000,
  "policyYears": 2
}
```

Responses

200 OK	Returns estimated PremiumAmount, IDV, and InvoiceAmount.
400 Bad Request	Invalid planId or missing fields.
401/403	Token missing or not Customer role.

Notes

- This is a read-only calculation endpoint. No VehicleApplication or Policy is created.

GET /api/Customer/my-policies

Get all policies belonging to this customer.

Auth Required: JWT required — Customer only

Returns all Policies where CustomerId matches the logged-in user. This is the main data source for the Customer Dashboard policy list.

Responses

200 OK	Returns array of Policy objects.
401/403	Token missing or not Customer role.

POST `/api/Customer/pay-annual/{policyId}`

Pay the annual premium for a policy.

Auth Required: JWT required — Customer only (own policies)

Records a payment for the current policy year. On success, sets IsCurrentYearPaid = true on the Policy. Creates a Payment record with a unique TransactionReference.

Path Parameters

policyId	int — PolicyId to pay for.

Request Body (JSON)

```
{
  "amount": 8500.00,
  "paymentMethod": 0,
  "transactionReference": "TXN2026031500123"
}

// PaymentMethod: 0 = Card, 1 = Net Banking, 2 = UPI, 3 = Wallet
```

Responses

200 OK	Payment recorded. IsCurrentYearPaid set to true on the Policy.
400 Bad Request	Payment already made for this year or invalid amount.
404 Not Found	Policy not found or does not belong to this customer.
401/403	Token missing or not Customer role.

POST `/api/Customer/renew/{policyId}`

Renew a policy for another year.

Auth Required: JWT required — Customer only (own policies)

Renews the policy by advancing CurrentYearNumber, updating CurrentYearEndDate, setting IsCurrentYearPaid = true, and setting IsRenewed = true. A new Payment record is created.

Path Parameters

policyId	int — PolicyId to renew.

Request Body (JSON)

```
{
  "amount": 8500.00,
  "paymentMethod": 2,
  "transactionReference": "TXN2026031500456"
}
```

Responses

200 OK	Policy renewed. CurrentYearNumber incremented. Payment recorded.
400 Bad Request	Policy already at max years (CurrentYearNumber = SelectedYears) or not eligible.
404 Not Found	Policy not found or does not belong to this customer.
401/403	Token missing or not Customer role.

Notes

- Cannot renew if CurrentYearNumber already equals SelectedYears — all purchased years are used up.
- Policy must be Active or recently Expired to be eligible for renewal.

POST `/api/Customer/claim/submit`

Submit a new insurance claim.

Auth Required: JWT required — Customer only

Creates a Claim against an active Policy. ClaimsService checks that the selected ClaimType is covered by the PolicyPlan before creating the record. ClaimCount on the Policy is incremented.

Request Body (JSON)

```
{
  "policyId": 7,
  "claimType": 0,
  "document1": "uploads/claim_damage_photo.jpg",
  "document2": "uploads/claim_repair_estimate.pdf"
}

// claimType: 0 = OwnDamage, 1 = Theft, 2 = ThirdParty
```

Responses

201 Created	Claim submitted. Returns new Claim object with ClaimNumber.
400 Bad Request	ClaimType not covered by the plan, or Document1 is missing.
404 Not Found	Policy not found or not active.
401/403	Token missing or not Customer role.

Notes

- Document1 is required. Document2 is optional but recommended.
- ClaimsService validates coverage: e.g., if claimType = 1 (Theft) but CoversTheft = false, the claim is rejected.
- ClaimCount on the Policy is automatically incremented on successful submission.

GET `/api/Customer/claims/my`

Get all claims filed by this customer.

Auth Required: JWT required — Customer only

Returns all Claims where CustomerId matches the logged-in user. Includes claim status, type, and decision details.

Responses

200 OK	Returns array of Claim objects.
401/403	Token missing or not Customer role.

POST `/api/Customer/policy/cancel/{policyId}`

Cancel an active policy.

Auth Required: JWT required — Customer only (own policies)

Sets the Policy Status to Cancelled and records the CancellationDate. Active Claims on the policy are not automatically closed.

Path Parameters

policyId	int — PolicyId to cancel.

Responses

200 OK	Policy cancelled. CancellationDate recorded.
400 Bad Request	Policy is not in a cancellable state (e.g., already Cancelled or Expired).
404 Not Found	Policy not found or does not belong to this customer.
401/403	Token missing or not Customer role.

Notes

- Cancellation is not reversible through the API.
- Any existing Claims linked to this policy are not automatically cancelled.

GET `/api/Customer/payments/my`

Get all payments made by this customer.

Auth Required: JWT required — Customer only

Returns all Payment records where the linked policy belongs to the logged-in customer.

Responses

200 OK	Returns array of Payment objects (Amount, PaymentDate, Status, PaymentMethod, TransactionReference).
401/403	Token missing or not Customer role.

POST `/api/Customer/transfer/initiate`

Initiate a policy transfer to another customer.

Auth Required: JWT required — Customer only

Creates a PolicyTransfer record with Status = Pending. The recipient customer will see this as an incoming transfer request and can accept or reject.

Request Body (JSON)

```
{
```

```
  "policyId": 7,
  "recipientEmail": "newowner@example.com",
  "newVehicleApplicationId": null
}

// newVehicleApplicationId is used when the transfer includes a new vehicle
```

Responses

201 Created	Transfer request created. Returns the new PolicyTransfer object.
400 Bad Request	Recipient not found, or policy does not belong to this customer.
404 Not Found	Policy not found.
401/403	Token missing or not Customer role.

Notes

- The recipient must be a registered user with Role = Customer.
- Transfer starts as Pending — the recipient must accept it for it to go through.

GET `/api/Customer/transfer/incoming`

Get incoming transfer requests for this customer.

Auth Required: JWT required — Customer only

Returns PolicyTransfer records where RecipientCustomerId matches the logged-in user. These are transfers that other customers have sent to this user.

Responses

200 OK	Returns array of incoming PolicyTransfer objects.
401/403	Token missing or not Customer role.

GET `/api/Customer/transfer/outgoing`

Get outgoing transfer requests initiated by this customer.

Auth Required: JWT required — Customer only

Returns PolicyTransfer records where SenderCustomerId matches the logged-in user. These are transfers this customer has sent to others.

Responses

200 OK	Returns array of outgoing PolicyTransfer objects.
401/403	Token missing or not Customer role.

POST `/api/Customer/transfer/{transferId}/accept`

Accept an incoming policy transfer.

Auth Required: JWT required — Customer only (must be the recipient)

Sets the PolicyTransfer Status to Approved and updates the Policy's CustomerId to the recipient's UserId. The Policy Status is also updated to Transferred.

Path Parameters

transferId	int — PolicyTransferId to accept.

Responses

200 OK	Transfer accepted. Policy now belongs to this customer.
400 Bad Request	Transfer is not in Pending status.
404 Not Found	Transfer not found or this user is not the recipient.
401/403	Token missing or not Customer role.

POST `/api/Customer/transfer/{transferId}/reject`

Reject an incoming policy transfer.

Auth Required: JWT required — Customer only (must be the recipient)

Sets the PolicyTransfer Status to Rejected and records a rejection reason. The original Policy stays with the sender.

Path Parameters

transferId	int — PolicyTransferId to reject.

Request Body (JSON)

```
{
  "rejectionReason": "I do not want to take over this policy"
}
```

Responses

200 OK	Transfer rejected. RejectionReason recorded.
400 Bad Request	Transfer is not in Pending status.
404 Not Found	Transfer not found or this user is not the recipient.
401/403	Token missing or not Customer role.

Error Reference

Here is a quick reference of all HTTP status codes used across the API and what they mean in VIMS.

Status Code	Status	When this happens in VIMS
200	OK	Request worked fine. Data or success message is returned.
201	Created	A new record was created (user, policy, claim, application, payment, transfer).
400	Bad Request	Something wrong with the request — CAPTCHA failed, validation error, email already used, business rule violated (e.g., ApprovedAmount too high, ClaimType not covered).
401	Unauthorized	JWT token is missing, expired, or invalid. User needs to log in again.
403	Forbidden	Token is valid but the user's role does not have access to this endpoint. E.g., a Customer trying to hit an Admin endpoint.
404	Not Found	The record doesn't exist or doesn't belong to the logged-in user (e.g., trying to access another customer's policy).
500	Internal Server Error	Something crashed on the server side. Check server logs.